

MPCA Module 2 — 3 Mark Questions & Answers

Q1. Explain the one-address and two-address instruction formats with examples.

1. One-Address Instruction Format:

- * Specifies an Opcode and **one explicit operand address**. The second operand is implicitly assumed to be in the **Accumulator (AC)** register.
- * **Syntax:** Opcode Address
- * **Example:** ADD X $\rightarrow$ Executes $AC = AC + Memory[X]$.

2. Two-Address Instruction Format:

- * Specifies an Opcode and **two explicit operand addresses** (Destination and Source).
- * **Syntax:** Opcode Destination, Source
- * **Example:** MOV AX, BX $\rightarrow$ Copies BX to AX ; ADD X, Y $\rightarrow$ Executes $Memory[X] = Memory[X] + Memory[Y]$.

Q2. Explain the fetch cycle with register transfer operations.

The **Fetch Cycle** reads the next instruction opcode from memory into the CPU using register transfer steps:

1. $MAR \leftarrow PC$ (Copy address from Program Counter to Memory Address Register).
2. $MDR \leftarrow Memory[MAR]$ (Fetch instruction byte from memory into Memory Data Register).
3. $IR \leftarrow MDR$ (Transfer fetched instruction opcode into Instruction Register).
4. $PC \leftarrow PC + 1$ (Increment Program Counter to point to the next instruction byte).

Q3. Differentiate between hardwired and microprogrammed control unit (any 3 points).

Feature	Hardwired Control Unit	Microprogrammed Control Unit
Design Approach	Hardware logic (gates, flip-flops, decoders).	Software logic (microprograms in ROM).
Execution Speed	Extremely fast (direct gate propagation).	Slower (requires ROM access time).
Modification / Flexibility	Difficult; requires physical circuit redesign.	Easy; requires updating microcode in ROM.

Q4. Explain the microinstruction format with its three fields.

A **Microinstruction Format** consists of three primary fields:



1. **Control Signals Field:** Binary bits that directly activate micro-operations (ALU lines, bus gates).
2. **Condition (CD) Field:** Specifies status flags (ZF, CF, SF) to evaluate for conditional branching.

3. **Next Address Field (ADR):** Specifies the Control Memory address of the next microinstruction to execute.
-

Q5. Explain bit-level and instruction-level parallelism with examples.

1. **Bit-Level Parallelism:** Increasing the word size of the processor (e.g., upgrading from 16-bit to 64-bit). Operates on 64 bits simultaneously in a single ALU cycle rather than multiple 16-bit steps.
 2. **Instruction-Level Parallelism (ILP):** Overlapping or executing multiple instructions concurrently within a single processor using **Pipelining** or **Superscalar** execution (e.g., executing an `ADD` and `MOV` in parallel).
-

Q6. Explain SISD and SIMD architecture with examples (Flynn's classification).

1. **SISD (Single Instruction, Single Data):**
 - * Traditional single-core processor. A single control unit processes a single instruction stream on a single data stream sequentially.
 - * **Example:** Intel 8086, classic single-core CPUs.
 2. **SIMD (Single Instruction, Multiple Data):**
 - * A single control unit issues one instruction, which is executed concurrently by multiple processing elements (ALUs) on distinct data vectors.
 - * **Example:** GPUs (Graphics Processing Units), vector processors, Intel AVX/SSE instructions.
-

Q7. Explain any two pipeline hazards with examples.

1. **Data Hazard (RAW):** Occurs when an instruction depends on the result of a preceding instruction that has not yet completed write-back.
 - * *Example:*
`ADD R1, R2, R3` (R1 written in Stage 5)
`SUB R4, R1, R5` (Tries to read R1 in Stage 2 before write-back)
 2. **Control Hazard (Branch Hazard):** Occurs when a conditional branch instruction changes the PC, making prefetched pipeline instructions invalid.
 - * *Example:* `JMP TARGET` causes prefetched sequential instructions to be flushed.
-

Q8. Explain the interrupt processing cycle briefly.

When an **Interrupt** is acknowledged at the end of an instruction cycle:

1. **Save CPU Context:** Pushes current Flags, Code Segment (`CS`), and Instruction Pointer (`IP`) onto the stack.
 2. **Disable Interrupts:** Clears Interrupt Flag (`IF = 0`) to prevent nested interruption.
 3. **Load ISR Address:** Uses the interrupt vector type to lookup the Interrupt Service Routine (`CS:IP`) address in the Interrupt Vector Table (IVT).
 4. **Execute ISR & Return:** Executes the handler, ending with `IRET` to restore Flags, `CS` , and `IP` from the stack.
-

Q9. Explain the basic 5-stage instruction pipeline (IF, ID, EX, MEM, WB).

1. **IF (Instruction Fetch):** Fetch instruction from memory using PC.

2. **ID (Instruction Decode):** Decode instruction and read operands from register file.
 3. **EX (Execute):** Perform ALU operation or calculate effective memory address.
 4. **MEM (Memory Access):** Read or write data operand from/to data memory (RAM).
 5. **WB (Write Back):** Write execution result back into destination register.
-

Q10. State the advantages and disadvantages of a microprogrammed control unit.

- **Advantages:**

1. High flexibility — instruction set changes require only updating microcode in ROM.
2. Simplifies implementation of complex CISC instructions.
3. Easy to debug and maintain.

- **Disadvantages:**

1. Slower than hardwired control due to Control Memory (ROM) read access latency.
 2. Higher manufacturing cost due to dedicated ROM chip.
-

Q11. Explain data hazards and their types (RAW, WAR, WAW).

Data hazards occur due to operand dependencies between overlapping instructions:

1. **RAW (Read-After-Write / True Dependency):** Inst 2 reads an operand before Inst 1 writes it.
 2. **WAR (Write-After-Read / Anti-Dependency):** Inst 2 writes to a location before Inst 1 reads its original value.
 3. **WAW (Write-After-Write / Output Dependency):** Inst 2 writes to a location before Inst 1 writes to it, leaving incorrect final data.
-

Q12. Explain why parallel processing is required (any three reasons).

1. **Overcoming Physical Limits of Clock Speed:** Physical heat dissipation and power consumption limit raw clock rate scaling.
 2. **Increasing Execution Throughput:** Allows multiple tasks/instructions to process simultaneously, delivering high performance.
 3. **Handling Compute-Intensive Applications:** Crucial for modern workloads like AI, Machine Learning, 3D graphics, and scientific simulations.
-